



65 V PRIMER

***Introduction to
Programming the 6502
via the
OSI 65V MONITOR***

OHIO SCIENTIFIC

© May, 1980

Contents

	Page
Introduction.....	1
Section 1 - Computers, Programs and The Monitor.....	2-4
Section 2 - Exploring Space with The Monitor.....	5-8
Section 3 - Creating a Good Listener.....	9-12
Section 4 - The Mighty Bit.....	13-15
Section 5 - The Ugly Truth About Binary.....	16-20
Section 6 - Hexaburger Helper.....	21-23
Section 7 - Making Up Orders From Headquarters.....	24-30
Section 8 - Making Your Move.....	31-35
Section 9 - Fetch and Step It.....	36-41
Section 10 - Start Waving Your Flag.....	42-46
Section 11 - A Total Mystery.....	47-50
Section 12 - A Loop You Can Count On.....	51-56
Section 13 - A Stack of Letters.....	57-60
Section 14 - It's The Same Old Routine.....	61-64
Section 15 - Extending The Mystery.....	65-70
Section 16 - Bit By Bit.....	71-74
Section 17 - It's Very Logical.....	75-78
Section 18 - Hardworking Subroutines.....	79-81
Section 19 - Two-Way Arithmetic.....	82-85
Section 20 - A Sparkling Finish.....	86-87
Appendices	
ASCII Character Codes.....	88
6502 Microprocessor Architecture.....	89
6502 Instruction Set	
a) Mnemonic List.....	90-91
b) Hex Listing.....	92-95
c) 6502 Disassembly Table.....	96
d) Special Symbols.....	97
e) Complete Instruction List with Opcodes.....	98-118
OSI 65V Monitor Mod 2 Listing.....	119-121
Bibliography.....	122-123
Index.....	124-125

Section Descriptions

Section 1 - Computers, Programs and The Monitor

Introduction of elementary computer terminology and concepts. Tells you what the Monitor is.

Section 2 - Exploring Space with The Monitor

An explanation of memory and how to examine and change your computer's memory with the Monitor.

Section 3 - Creating a Good Listener

You encounter your first "machine code" program, "Good Listener", in this section and get an introduction to flowcharts.

Section 4 - The Mighty Bit

Binary and the ASCII code are introduced and "Good Listener" is expanded.

Section 5 - The Ugly Truth About Binary

More binary, the introduction of BCD and hex.

Section 6 - Hexaburger Helper

Arithmetic in the world of hex.

Section 7 - Making Up Orders From Headquarters

Introduction of registers, flowchart explanation of "Good Listener". Introduction of opcodes, addressing modes, mnemonics and assembly language.

Section 8 - Making Your Move

Data moving instructions and associated addressing modes.

Section 9 - Fetch and Step It

The Program Counter, Instruction Register, branch and jump instructions and zero page are introduced. "Execute-in-a-Box" program.

Section 10 - Start Waving Your Flag

Flags, compare instructions are covered.

Section 11 - A Total Mystery

A "mystery program" presented, flow charted and explained. Two's complement introduced.

Section 12 - A Loop You Can Count On

Loops are discussed along with increment and decrement instructions.

Section 13 - A Stack of Letters

Stack, push, pull and stack pointers are examined. "Execute-in-a-Box" is reviewed.

Section 14 - It's The Same Old Routine

Disassembly table and subroutine concepts are covered.

Section 15 - Extending The Mystery

"Total Mystery" is explained. Add and subtract instructions and carry flag introduced.

Section 16 - Bit By Bit

Arithmetic and logical shifts and rotate instructions.

Section 17 - It's Very Logical

Masking bits, logical operation (AND, OR and EOR) discussed.

Section 18 - Hardworking Subroutines

Some handy subroutines.

Section 19 - Two-Way Arithmetic

Decimal mode and relocatable code.

Section 20 - A Sparkling Finish

Indexed Indirect Addressing Mode and a sparkling program.

Introduction

Your Ohio Scientific microcomputer is in many ways like the computers that write company paychecks and run factories, control space missions and predict the weather. This manual is to show how your microcomputer and its big brothers work.

It is not assumed that you know anything about computers already, just that you want to know the inside story. You will be discovering how to use your computer at a very direct level, one that permits you to control every capability that it has. Convenient access to this level is available through a part of your system known as the 65V Machine Monitor. The first section will tell what the Monitor is. The remainder of the manual will guide you in using the Monitor to explore and control your computer. There will be twenty sections. Each section will discuss a new topic and will suggest something new for you to do with your system that will help you to understand computers.

Take your time and play around with the new elements introduced in each section before going into the next section. Enjoy yourself.

1. Computers, Programs and the Monitor

Computers are information processing devices. That is their thing! They accept input information and transforms it into output data that we can read and interpret, or into sequences of actions which we find useful or pleasing.

Not all information processing takes place in computers. One drives a car by giving it the necessary input through the ignition, steering wheel, accelerator, etc. The result is a mechanical amplification of the input information. But, a car is not a computer. There is but one way to process information through a car and that is to drive it. Social benefits aside, it is a single-purpose device.

A computer can be redirected easily to different information processing tasks. It is a general-purpose device. Switching tasks is easy because the description of a task is itself information, the natural food for a computer. Of course, the computer is a machine without intelligence, so the input must be put into a completely prescribed form, leaving nothing to be figured out on the basis of previous experience.

Computer input is of two kinds: the information to be processed and the directions for the processing. Information to be processed is called input data. Several kinds of input data will be introduced later. The directions for processing take the form of a sequence of actions called a program. The actions to complete a task may be called an algorithm. Expressing the algorithm in a form suitable for computer input, we have a program. The algorithm must be described in a

language that can be "understood" by the computer, in the sense that the computer takes the right action.

Languages developed for this purpose are called computer languages. One type of computer language spells out the precise way in which each part of the computer is to participate in processing the data. This is the machine language of the computer. Since the parts of different computers are arranged differently, their machine languages are different.

When carrying out a task by executing the program which describes it, a computer is actually processing a machine language program. This does not mean that people who develop programs (programmers) always write their programs in machine language. After all, the translation of a program from another computer language into machine language is just another information processing task for the computer. When a translation is being done, the input computer language is called the source language or source code. The source language is translated by the computer into the object language. Since the objective of the translation is usually machine language, the term object language or object code is often used for machine language.

The 65V Machine Monitor is a machine language program. We will refer to this by using just the term Monitor. It does not have to be fed to your computer as input because it is built-in. Built-in programs control your computer from the time it is switched on, until the control of the computer is given to a program you have entered as input information. The program in control at start-up places a "prompting" message on the video display requesting that you select

which built-in program to be executed next. One of the options in the prompting message is an "M". The program is reading the keyboard for input over and over, just waiting for you to type something.

As you depress the "M" key, control of the computer is turned over to the 65V Machine Monitor. The transfer to the Monitor program occurs before you can release the "M" key. The Monitor writes a message on the video display, to be explained in the next section, and begins to read the keyboard, searching for your next input.

Make sure the 'SHIFT LOCK' key on your keyboard is depressed. Reset the computer (this method varies between different OSI computers - consult the operators' manual for your particular computer). Type "M" in response to the displayed message. This starts execution of the 65V Machine Monitor.

Observe the display while depressing various keys and try to determine which keys the Monitor interprets as commands. You'll find the answer as you read on. Two commands 'G' and 'L' terminate the execution of the Monitor. Avoid them until you are finished playing hide and seek. Soon you will be entering and executing machine language programs with the aid of your humble and ever present servant, the 65V Machine Monitor.

2. Exploring Space with The Monitor

Your computer is a system consisting of physical machinery controlled by stored information in the form of programs and data. The physical configuration does not change as programs execute. That part of the system is called hardware - the circuit boards, metal cases, etc. Computer programs are frequently changed and so they are collectively referred to as software. The hardware components of your system are represented in Figure 2.1.

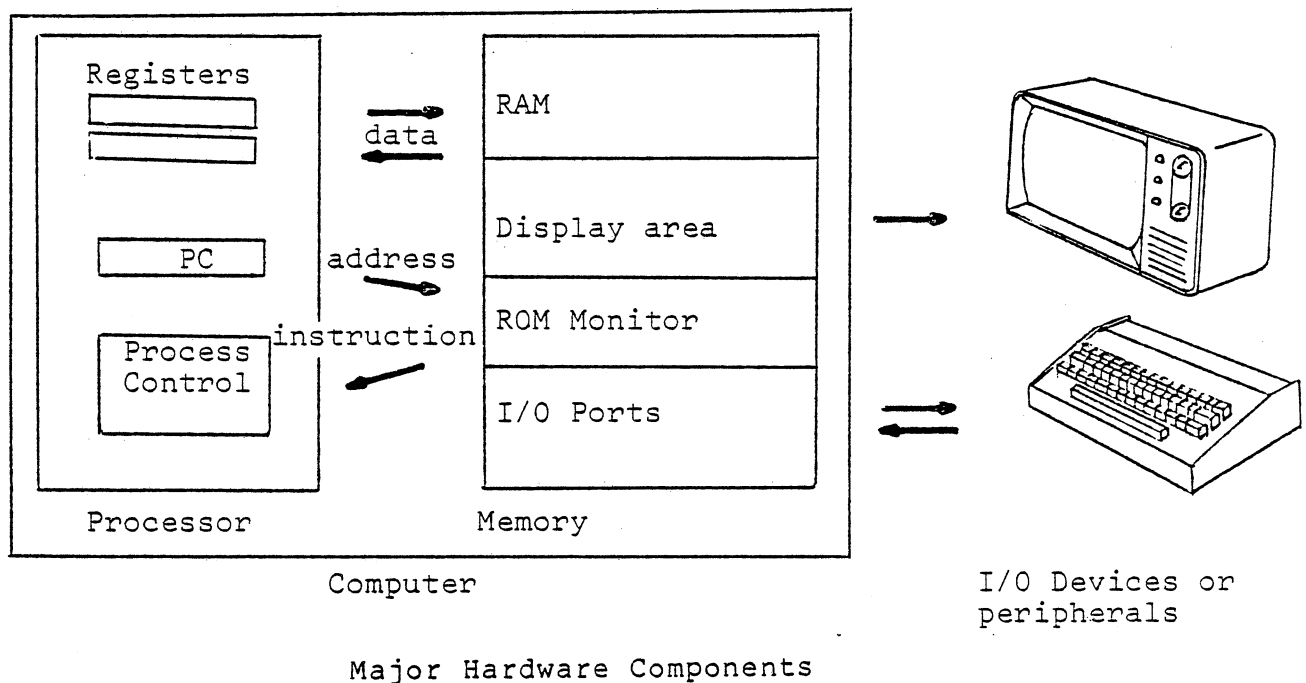


Figure 2.1

The programs in control of the system are stored in the memory. A memory consists of a large number of cells. The contents of a cell are made available to other components by means of its address, a unique number associated only with that cell. One cell contains a piece of information we call a byte. The byte is defined in Section 4. Cells are also called memory locations.

The "action" component that processes data from the memory is the processor. In your system, the processor is a 6502 microprocessor. Information storage locations and processing stations within the processor are called registers. A single action that a processor can take with some register is called an instruction. A machine language program is a series of instructions. When one of the processor's registers, the program counter (PC), is set to the address of the beginning cell of a program, the processor begins to read and execute the machine language instructions contained in those cells. The resulting execution of the program causes some memory cells to be altered and exchanges of information to take place between the memory and input/output (I/O) devices. The result is the activity of the computer system that you can observe, such as a loan calculator or video animation.

Memory cells are not all alike. Some portions of memory hold fixed information that cannot be altered as programs are executed or when the computer is turned off. This portion is called Read Only Memory (ROM). The 65V Machine Monitor and other built-in programs are located in ROM. Other parts of memory are for temporary information, like the data being processed by executing programs, or programs which are brought into memory to produce a desired behavior at a particular time. This kind of memory is known as Random Access Memory (RAM) or read/write memory.

As you start the execution of the 65V Machine Monitor, the processor clears the CRT screen (video display) and writes a four-digit number and a two-digit on a small portion of the display.

This is done by transferring characters into the RAM display area as shown in Figure 2.1. Contents of this RAM display area are regularly transmitted to the CRT display screen.

The characters appearing on the screen represent two numbers written in a number system called hexadecimal (base sixteen). In a section to follow you will learn how these numbers are constructed. The left number is the address of a memory cell and the right number shows the information contained in that cell. Addresses (or memory cell numbers) are two bytes long and are represented by four hexadecimal digits. Two digits represent a byte, the contents of one memory cell. For reasons we can appreciate shortly, the Monitor ignores most keys reacting only to the decimal digits 0-9, the letters A-F and L, the special symbols: '/' (slash), '.' (period), and the 'RETURN' key.

The decimal digits 0-9, along with the letters A-F are hexadecimal digits. The Monitor has two ways to react to keyed in hexadecimal digits. In the address mode, digits are rolled into the address value (left or four-digit number) displayed. This changes the right value too, since the new address designates a different memory cell. In data mode, digits are rolled into the data value (right or two-digit number). Actually, the addressed memory cell is being altered and the display change reflects these changes.

The Monitor starts in the address mode. Depressing '/' changes it to data mode. When '.' is depressed, the Monitor returns to address mode. The 'RETURN' key advances the address by one, changing the data value. You can use it to move through memory, reading or writing into

cells.

The 65V Machine Monitor program starts at a location whose address is \$FE00 (F, E, zero, zero - the \$ indicates that this is a hex value). See if you can get that number into the address side of the display. This displays the first instruction of the Monitor on the data side. It should be 'A2'. Now depress '/' and the 'RETURN' key several times. What is happening? You are counting up on the address side and getting a display of successive cells on the data side. Wherever you stop, write down the address and write '22' into that cell using the data mode. Now return to address mode ('.') and key-in the address over itself. Disappointed? The '22' is not there because you are addressing ROM, where the contents of cells cannot be changed.

Can you enter data and see the result immediately? Yes, in the display area. Each character position on the CRT screen has a RAM cell associated with it which you can alter in data mode. Most of them now contain the 'blank' character. Given the address of a cell in the display area, you can place non-blank characters in nearby cells to create a pattern on the screen. When you learn how to count in hexadecimal, you can systematically explore the display area and find out exactly where screen positions are recorded in the display memory. A starting location for some preliminary exploration is the hexadecimal number \$D140, a strange looking number if I ever saw one.

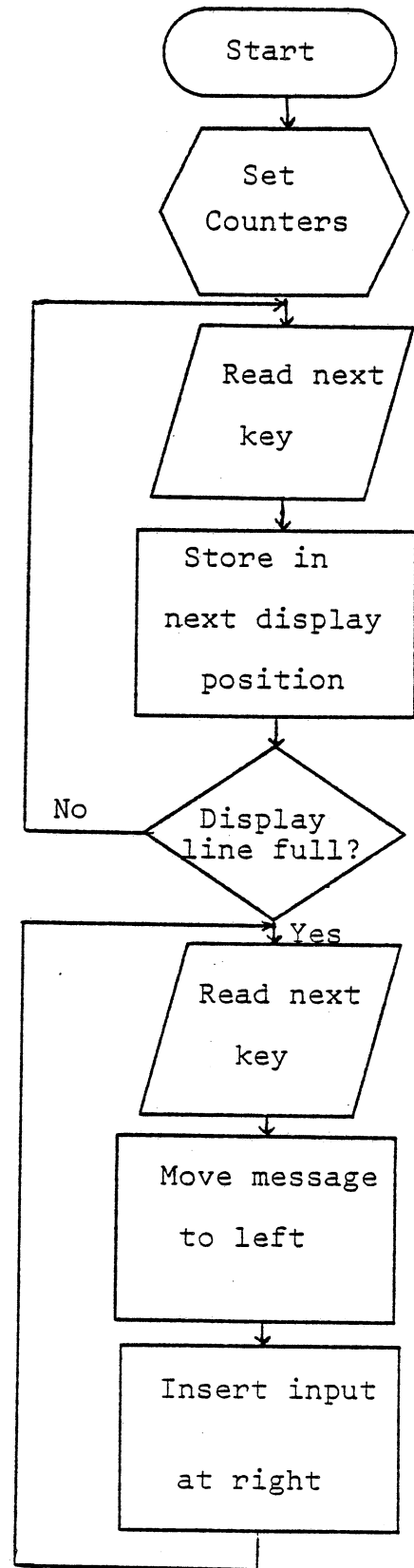
3. Creating a Good Listener

It may be that good listeners are born, not made. But your computer can be programmed to pay strict attention to every character you type. It cannot nod its head in agreement but it can repeat or echo each character on the television screen.

Let's think about the program which describes the "good listener" task. In order to write a program we have to specify the desired behavior completely, leaving nothing unspecified. The computer understands nothing and must be told everything. For starters, exactly where on the screen do we want the input to be echoed? Where the Monitor prompt message was? Okay. When our program is executing, the Monitor will be asleep, having no processor to execute it, so these display locations are available for display. Let's extend the display across the screen so we can read the glorious stuff we shall be entering. What happens when we reach the right of the screen? Later you can implement other choices, but for now, let's start pushing characters to the left making room for new input to the right, just as the Monitor does to its address and contents display. Will there be any keys which will be used to cause the "good listener" to alter its behavior, like the '.' and '/' commands affect the Monitor? No, not in this 'first pass' version. How will the good listening end? We can't tell it to stop so let it go on forever! "Forever" means as long as the program has control of the processor.

Computers do one thing at a time, so a program must be organized that way. An important tool for planning or showing the sequence of steps in a program is a flowchart, a diagram such as the one to the right. Arrows in a flowchart mark the path of the processor, boxes represent the processing that it does along the way. Box shapes have meaning to programmers. Rectangular boxes stand for processing steps. Slanted sides signal an input or an output. Diamonds are decision boxes, representing tests which select paths for the processor. In the program a decision step is called a branch. Branches allow sets of steps to be repeated and decisions to be made. A set of repeated steps is called a loop. The six-sided box near the top denotes preparation for a loop. Ovals show the start and termination points of the program.

You probably have realized that the flowchart represents the "good listener". There are two loops, one to fill the display line and another to add characters to a full line. The lower loop has no decision box, therefore, there is no way for it to end. There is usually one start point and there can be several termination points.



A Flowchart

Figure 3.1

The best way to see what the program does is to try it. The sequence of steps in the flowchart must be expressed in computer language. This programming step is called coding. Coding in machine language requires the programmer to express the required processing in terms of operations which the processor can do with its registers. Using techniques you will be learning in this manual, a machine language version of the "good listener" was created in hexadecimal numbers, so that you can load it from the keyboard. It should be loaded starting at location 0000, as shown. All key-in's for loading, using the monitor, are shown below. We use the symbol '*' to represent the 'RETURN' key. Locations are shown to the left for checking during loading.

<u>Location</u> (hex)	<u>What You Should Key-In</u>
0000	.0000/A2*00*20*ED*FE*9D*46*D1*
0008	E8*E0*14*D0*F5*20*ED*FE*
0010	A8*A2*00*BD*47*D1*9D*46*
0018	D1*E8*E0*13*D0*FE*98*8D*
0020	59*D1*4C*0D*00

Before executing the program let's check the loading. One key-in error can make a big difference. How do we check it? Type ".0000/" - this resets the Monitor to the data mode and resets the display to the beginning of the program (\$0000). Repeated use of the 'RETURN' key allows you to "step" through the program examining each location. If you find a mistake, enter the correction.

For example, suppose you type ".0000/" and depress the 'RETURN' key three times. Suppose the display reads "0003 DD", this is wrong, it should be "0003 ED". To enter the correction, simply type the correct contents, E followed by D. Note that the display now reads "0003 ED" which is correct. Your correction is now entered. Continue with the 'RETURN' key to check the code. Repeat the above process as necessary until the code on Page 11 is entirely correct.

Now that the program is loaded and checked, it can be executed. The 'G' command tells the Monitor to load the displayed location into the program counter, thus, starting the execution of the program beginning at the displayed cell. So, .0000G will start the "good listener". Is it listening?

4. The Mighty Bit

Information comes into the computer system through its input/output (I/O) devices and is stored in memory. It moves around inside the processor. Output information appears on the display as numbers, words or pictures. But if we were inside the computer watching the information go by, we would see it in a strange internal form.

A computer is made of electrical circuits and connecting paths which hold, transmit or process the smallest particle of computer information there is, the bit. A bit is a simple yes or no, a true or false, one of two possible values. Internally, this is seen as a one or a zero. It is the universal measure of the amount of information content in any message. A set of bits taken together is a binary code. Just as a bicycle has two wheels, a binary code is based on two values. Like the parallel rails of a railroad track, sets of electrical paths carry binary codes around inside the computer.

Anything that can be said, can be said in a binary code, provided there are enough bits and the bit values have been assigned a meaning. The bits themselves do not carry the meaning. Sitting there in the computer, we would not get any message by reading a passing binary code unless we knew the meaning assigned to the bit values by the sender and receiver of the code.

To represent a binary code on paper, we write it as a string of characters in which each bit is represented by one of two possible characters. Bit values are traditionally represented by the symbols

'0' (zero) and '1' (one).

A byte is a binary code of eight bits. In your computer each memory cell contains one byte. A character, such as a letter, digit, or punctuation mark, is the type of information which is (forgive the pun) byte size. When reading the keyboard, the processor loads a byte from the keyboard into one of its registers. This byte indicates which key is being depressed. "Good listener" stores each byte it obtains in memory cells in the display area. A byte in the display area directs the display subsystem to a pattern for the character which is to be seen at that position.

Somebody had to decide what bit pattern should represent each character. Many computer manufacturers use the same code, so that computers can transmit character information to each other. The common code used in almost all small computers is the American Standard Code for Information Interchange or ASCII code.

You can look up the ASCII code for any keyboard character in the table in the appendix at the rear of this manual. We have another way to determine the ASCII value - we can execute a program which reads the keyboard, converts and displays the bit values it is receiving. The load and execute Monitor commands for this conversion program are:

```
.0000/20*ED*FE*85*F0*A9*18*A2*
```

```
07*9D*12*D1*66*F0*3E*12*
```

```
D1*CA*10*F5*30*EA
```

```
.0000G
```

As you examine ASCII codes, go through the sequence of digit characters 0,1,...,9 and note what you see.

If it is convenient to leave your computer on while reading the next section, it will save time. The next section suggests a small change in the binary display program, which can be made without keying in the program again, provided power has not been removed from the RAM memory in which you are entering programs.

5. The Ugly Truth About Binary

Numbers are important to computers. Addresses of memory cells are numbers. The contents of registers and memory cells often represent numbers. Computers spend a lot of their working time counting and calculating with numbers. So, numbers have to be represented in binary codes, don't they? One way of doing it could be seen in the ASCII codes for digits 0-9. Ignoring the left four bits which remain fixed at 0011, the digits are represented this way:

0 - 0000	5 - 0101
1 - 0001	6 - 0110
2 - 0010	7 - 0111
3 - 0011	8 - 1000
4 - 0100	9 - 1001

To represent a number like 6502, we can string these codes together, packing two-digit codes to a byte like this:

6502 = 01100101 00000010
 6 5 0 2

This is a frequently used code for numbers. It is called Binary Coded Decimal or BCD. Can you write the four-bit codes that are not used in BCD for digits? There are six of them*. These unused codes are the reason why BCD is not the most efficient way to represent numbers in a computer. A memory cell containing a BCD byte is storing only 5/8 of

*The unused codes are: 1010, 1011, 1100, 1101, 1110 and 1111.

the information it could, because its bits are not free to express all of their values. Also, the processor circuits which carry out such pleasing operations as addition and subtraction are complicated by the need to avoid the codes that do not represent digits.

So what is a "natural" number system for computers? One that will allow them to do their thing? We humans are partial to tens. We write the digits 6502 to mean:

$$(6 \times 1000) + (5 \times 100) + (0 \times 10) + (2 \times 1)$$

$$\text{or } (6 \times 10^3) + (5 \times 10^2) + (0 \times 10^1) + (2 \times 10^0)$$

The decimal digits 0,1,...,9 express numbers in the decimal number system based on the number ten. What's so great about ten? Take off your mittens.

If you are a computer, how many fingers do you have? Two. The characters '0' and '1' stand for your digits and the natural way for you to express the above number is 1100110010110, meaning:

$$\begin{aligned} & (1 \times 2^{12}) + (1 \times 2^{11}) + (0 \times 2^{10}) + (0 \times 2^9) + (1 \times 2^8) + (1 \times 2^7) + (0 \times 2^6) \\ & + (0 \times 2^5) + (1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \end{aligned}$$

This is the binary number system based on the number two. The BCD codes for numbers up to ten are the binary representations of these numbers.

In the binary number system every bit pattern is used and stands for a unique number, but only a computer could love a number like 1100110010110. The ugly truth about binary is that it scrambles human brain waves. We lose our place in its maze of zeros and ones. The human brain deals best with a few things at a time. We even put commas in long decimal numbers to mark the place. What we need to deal with binary is something to group those bits into a shorter, more readable pattern.

The answer lies in those six missing four-bit codes in BCD. If we had digits to represent those extra codes, we could write binary compactly because every four-bit group could be replaced by the corresponding digit. A digit is some character given a numerical meaning. To get six more, we use the letters A-F and assign numbers and their four-bit binary representations to them, arriving at:

0 - 0000	4 - 0100	8 - 1000	C - 1100
1 - 0001	5 - 0101	9 - 1001	D - 1101
2 - 0010	6 - 0110	A - 1010	E - 1110
3 - 0011	7 - 0111	B - 1011	F - 1111

The number system based on sixteen is called hexadecimal. If our explorations of the universe reveal intelligent life forms with sixteen fingers, they probably will be using hexadecimal.

To see how that ugly binary number 1100110010110 shapes up in its hexadecimal form, just mark off groups of four bits from the right 1/1001/1001/0110 and replace each group by its hexadecimal digit. You should get 1996₁₆.

Another system that is sometimes used to represent binary in computer literature is the system based on eight. It is called octal. In octal, the digits are zero through seven and each digit replaces three bits.

With all these number systems floating around we could get confused. When there is any question about it, the base number of the system is written as a subscript of the number representation. For example:

$$6502_{10} = 1100110010110_2 = 1996_{16}$$

How would our 16_{10} -fingered friends write that? Answer:

$$6502_A = 1100110010110_2 = 1996_{10}$$

Can you figure out why?

You can enlist your computer's help in a little binary to hexadecimal drill by making a small modification in the binary display program of the last section. The program loops forever, always looking for a new key-in. Let's make it return to the Monitor after accepting one key-in. Then you can use the Monitor to place in a display cell what you think is the hexadecimal version of the binary code that you see. If you are right, the character originally keyed in will appear on the screen.

If you haven't turned your computer off since entering the binary

display program, it is still there in RAM! Going to the Monitor, alter the program by entering:

.0014/4C*2A*FE*

Now when you enter .0000G, the modified program will display in binary the ASCII code for your next key-in. Select a nearby display location and enter the hexadecimal code. If you make a mistake on the binary to hexadecimal conversion, something other than the character you expected will appear, maybe something you cannot key-in. Does that make you curious? Then why not make some "intentional" mistakes? Who'll know?

6. Hexaburger Helper

So why does an alien being with eight fingers to the hand (or four fingers on each of four hands) write

$$6502_A = 1100110010110_2 = 1996_{10}$$

with his little alien BIC? Because to him 10 is the one-digit number A. And to him 10 means sixteen. Of course, he would never call it "sixteen". That's decimal-based terminology. In any number system, 10 stands for the number on which the system is based. This number is referred to as the base or the radix of the system. Remember that there is just one set of integer numbers but many systems with which to express them. The equal signs in (back to earth)

$$6502_{10} = 1100110010110_2 = 1996_{16}$$

belong there because we have three representations of the same number.

To find the decimal representation of a hexadecimal number, say, $D1E4_{16}$, you translate its digits to decimal and carry out the decimal calculation it stands for, namely

$$(13 \times 16^3) + (1 \times 16^2) + (14 \times 16^1) + 4 = 53732$$

The calculation is a bit easier if you alternate multiplications and additions like this:

$$((13 \times 16 + 1) \times 16 + 14) \times 16 + 4 = 53732$$

Written out, the calculations look like this:

$$\begin{array}{r}
 13 \\
 \times 16 \\
 \hline
 78 \\
 130 \\
 \hline
 208
 \end{array}
 \quad
 \begin{array}{r}
 208 \\
 + 1 \\
 \hline
 209
 \end{array}
 \quad
 \begin{array}{r}
 209 \\
 \times 16 \\
 \hline
 1254 \\
 2090 \\
 \hline
 3344
 \end{array}
 \quad
 \begin{array}{r}
 3344 \\
 + 14 \\
 \hline
 3358
 \end{array}
 \quad
 \begin{array}{r}
 3358 \\
 \times 16 \\
 \hline
 20148 \\
 33580 \\
 \hline
 53728
 \end{array}
 \quad
 \begin{array}{r}
 53728 \\
 + 4 \\
 \hline
 53732
 \end{array}$$

The result: $D1E4_{16} = 53732_{10}$.

As you work with machine language programs, you will want to add two hexadecimal numbers and get a hexadecimal result. One way is to convert the numbers to decimal representation, add them, then convert the sum back to hexadecimal. That's the hard way. The easy way is to add in hexadecimal, using the same rules of arithmetic we use in decimal addition. Sounds hard, doesn't it? Well, judge for yourself. First, let's take a close look at what's happening in the decimal addition:

$$\begin{array}{r}
 1 \\
 182 \\
 + 137 \\
 \hline
 319
 \end{array}$$

Since $8 + 3$ cannot be represented in one decimal digit, you think of it as $10 + 1$ and write the 1, noting the carry into the column to the left, as shown.

Hexadecimal addition works on the same way. Write the digits of $B6_{16} + 89_{16}$ in the format for column-by-column addition, then add the units column digits. The answer, 15_{10} , is represented as F in hexadecimal. In the sixteen's column, we get $11 + 8 = 19_{10}$, which cannot be represented in one hexadecimal digit. So, we think of it as $16 + 3$, write down the 3, and carry the 1 from $16 = 10_{16}$. The whole problem then reads:

$$\begin{array}{r} B6_{16} \\ + 89_{16} \\ \hline 13F_{16} \end{array}$$

Hexadecimal subtraction can be done directly, following the same rules as decimal subtraction: In

$$\begin{array}{r} 13F_{16} \\ - B6_{16} \\ \hline 89_{16} \end{array}$$

we must borrow 1 to go with 3. But, the borrowed 1 stands for 16_{10} , so we calculate $(16 + 3) - 11 = 8$.

Hexadecimal addition and subtraction are a help in finding out exactly where display positions appear on your screen. The number of display positions in a row is a multiple of 16_{10} . It may be $32_{10} = 20_{16}$ or $64_{10} = 40_{16}$. This means that locations displayed in a vertical column differ by that multiple.

7. Making Up Orders From Headquarters

Remember the flowchart for "good listener"? It showed the sequence of steps for doing the desired processing and display task. Such a sequence of steps is called an algorithm, as we mentioned earlier. To bring the "good listening" algorithm to life in your computer, you loaded a machine language program into memory as a sequence of binary codes, keying in each byte of the program as a pair of hexadecimal digits. How did "good listener" get into that form? By a process that will be described in this section, a process involving two stages called coding and assembly.

In the coding stage, the programmer expresses his chosen algorithm as a series of actions that the computer's processor can perform. Each individual action is an instruction. A processor has a menu of available instructions called the instruction set. Working from a flowchart or some other description of the algorithm, the programmer chooses sequences of instructions. These include branch instructions which select sequences to be executed next and other instructions which cause some sequences to be executed repeatedly.

The structure of the microprocessor determines what instructions are available in the instruction set. Coding in machine language requires a knowledge of that structure because you are expressing the algorithm directly in machine instructions.

Translation of algorithms into more general computer languages (also called higher level languages) is also called coding. This approach does not require the programmer to know the "insides" of the

processor. This knowledge is applied in a program called a compiler or an interpreter that translates the higher level computer language into machine language instructions. Some high level languages that are translated into machine language are BASIC, FORTRAN, PASCAL, APL and COBOL.

We are going to be coding in the machine language of the 6502 microprocessor. The 6502 has three registers for which programmers use the names X, Y and A. Each of these registers holds one byte of information. The accumulator A is the primary processing register by virtue of its direct connections with the processing circuitry in the 6502. The X-register and Y-register are called indexing registers. Their usual jobs are counting and maintaining the location (or index) of data being processed.

A translation of the "good listener" flowchart into operations on the contents of 6502 microprocessor registers is given in Figure 7.1. To the right of each box is a description of a way to use the 6502 registers to carry out that step. The programmer may have a choice of several possible ways to use the registers. Of course, Figure 7.1 does not represent the completed coding because the instructions are not specified. Besides spelling out the instructions, the coding step also involves specifying the location and arrangement of data. In the "good listener" coding, for example, access to display locations is done by incrementing the X-register (register X) by one each time a loop is repeated. This approach dictates the arrangement of that data.

In the coding stage, some flowchart boxes may be implemented by one or two instructions. Others may take many instructions. In

Load a zero into Register X.

Use a part of the Monitor which prints the ASCII code for the next key depressed into the accumulator A.

Store from the accumulator A into the memory cell whose address is $D224_{16}$ plus the contents of X.

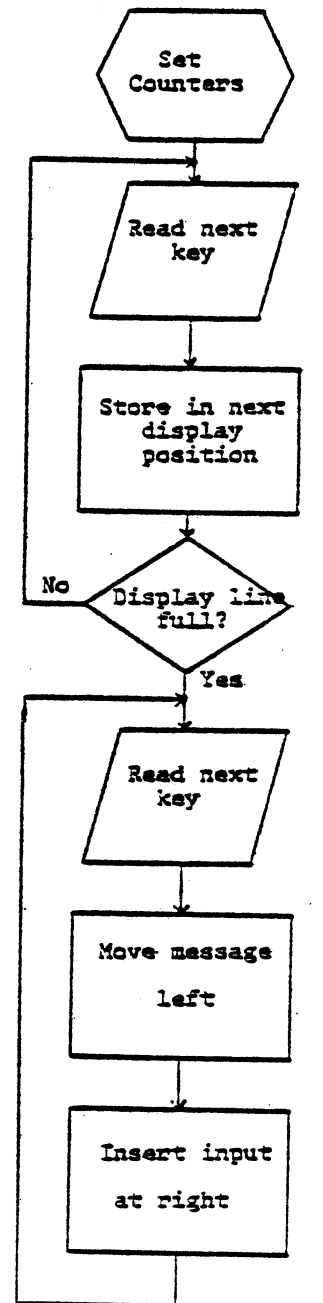
Increment X by 1 for next store.

Compare X with length of display line. If not yet equal, branch back for next key.

Use the same routine to get ASCII code for next key depressed into the accumulator.

Load a zero into Register X. Repeat the following loop across the display: load from the contents of $D225_{16}$ plus X into Y, store from Y into $D224_{16}$ plus X. Increment and test X.

Store from the accumulator into the address on the right of the display line. Jump back to the instruction for reading a key.



"Good listener" as
Register Operations

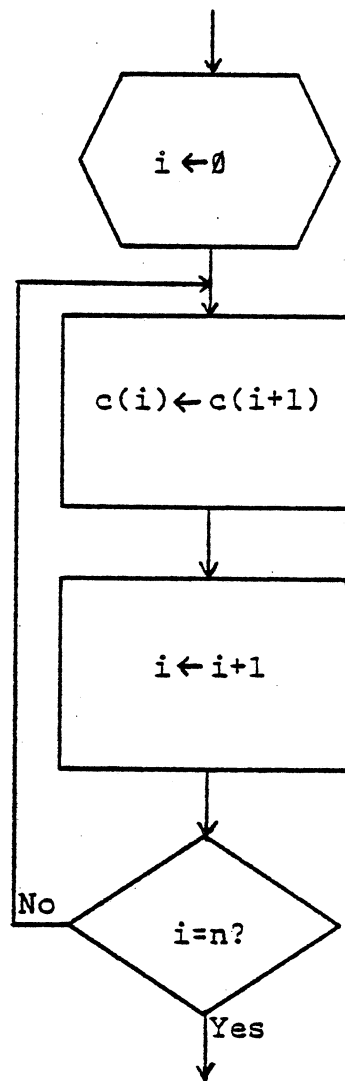
Figure 7.1

describing the "good listener" algorithm, it is helpful to use a single box for the "move message left" step. Implementing this step with register operations requires a loop, as shown in Figure 7.2. Loops often involve incrementing and testing an index, such as *i* in Figure 7.2. The index must be given an initial value before the loop is executed the first time.

Now getting down to selecting instructions, just what does an instruction contain? An instruction always contains a binary code called an operation code, or opcode, for short. The opcode identifies the operation to take place and the manner in which the operands, the data involved in the operation, are to be accessed. Operands are contained in either microprocessor registers or in memory cells. In the case of memory cells, the instruction selects one of several addressing modes for the operand. Addressing modes are ways of forming the memory address of the operand. In addition to the opcode, instructions often contain the operand address or information that goes into the formation of the address. The operand address derived during the execution of the instruction is called the effective address.

Figure 7.3 shows a coding of "good listener". The symbolic form of the program that most programmers use in coding is under the heading Mnemonic. The numeric form of the same program that results from the process of assembly is under the heading Opcode.

Just as binary numbers are hard to interpret, machine language programs in binary form are not easy to deal with. Since programmers want to be able to read and alter programs as well as create them,



A loop for "Move message left"

Figure 7.2

<u>Location</u>	<u>Opcode</u>	<u>Operand</u>	<u>Label</u>	<u>Mnemonic</u>	<u>Operand</u>	<u>Remark</u>
0000	A2	00		LDX	#0	; CLEAR INDEX
0002	20	ED FE	FILL	JSR	GETKEY	; NEXT PRESSED TO A
0005	9D	46 D1		STA	\$D146	; INTO NEXT LINE CELL
0008	E8			INX		; INCREMENT INDEX BY 1
0009	E0	14		CPX	#20	; END OF THE LINE?
000B	D0	F5		BNE	FILL	; BACK UNTIL EQUAL
000D	20	ED FE	REPEAT	JSR	\$FEED	; NEXT PRESSED TO A
0010	A8			TAY		; SAVE KEY IN Y
0011	A2	00		LDX	#0	; CLEAR INDEX
0013	BD	47 D1	MOVE	LDA	\$D147	; LOAD LINE(I+1)
0016	9D	46 D1		STA	\$D146	; STORE INTO LINE(I)
0019	E8			INX		; INCREMENT INDEX
001A	E0	13		CPX	#19	; END OF THE LINE?
001C	D0	F5		BNE	MOVE	; BACK UNTIL EQUAL
001E	98			TYA		; RESTORE KEY-IN
001F	8D	59 D1		STA	\$D159	; STORE NEW KEY
0022	4C	0D 00		JMP	REPEAT	; BACK FOR MORE

Good Listener as Machine Language Instructions

Figure 7.3

they usually write them in a symbolic form using names for operations, registers and memory locations. There is also the important matter of getting the new program to work in the first place. This process is called "debugging". There always seem to be some little mistakes or "bugs" in a new program, and finding them requires concentrated study of the program. So a readable form of the program is essential.

Assembly is the process of translating the symbolic form of machine language into binary codes. If the symbolic form of the program is written according to required format rules a program can do the assembly task. Such a program is called an assembler. The input to the assembler is called assembly language. The symbolic form of Figure 7.3 and other such figures in this manual conform to the standard assembly language mnemonics (symbols) for the 6502 (see list in appendix).

In later sections we shall return to Figure 7.3 to learn about every part of it. For the moment, see if you can tell where to change the length of the display line. Compare the assembly language version with the flowchart and description of Figure 7.1. Every instruction is on a separate line and the assembled instruction begins with the opcode. In the assembly language, the numbers relating to display line length are in decimal; the corresponding numbers are in hexadecimal in the assembled code. Can you find them? Now select another line length that will fit on your screen, alter the program and try it. A clue: there are three bytes in the program that must be altered.

8. Making Your Move

How is coding a program like sorting potatoes? Answer: It's just one decision after another. Would you like to understand the process and do some coding yourself? Okay, let's examine some of the decision-making that takes place in coding. Mostly, the "good listener" algorithm involves moving data around. For now, we'll concentrate on decisions connected with that.

The part of the 6502 instruction set devoted to moving data around between memory and processor registers X, Y and A appears in Figure 8.1. There you have almost all you need to know about these instructions for coding and assembly. Missing is the time it takes the processor to execute each instruction. We are seldom concerned with that. Many manuals and books provide 6502 instruction tables with execution timing, see the bibliography for references.

Starting with the leftmost column of Figure 8.1, the operation mnemonic is a name for the operation, a name which is suppose to be easy to remember. Compare the mnemonics to the explanations of the operations and judge that for yourself. Next we have opcodes for each of the addressing modes available with that operation. The opcode determines both the operation and addressing mode. Not all combinations of operation and addressing mode are available in the instruction set.

At the right there is an indication of any flags affected by the operation. This is included here for the sake of completeness. Flags are one-bit registers in the processor whose functions we will go into

Opcodes for Various Modes

Address Modes

<u>Mnemonic</u>	<u>Explanation</u>	immediate	absolute	zero page	(ind,X)	(ind),Y	zero page,X	abs,X	abs,Y	zero page,Y	<u>Flags</u>
LDA	load A	A9	AD	A5	A1	B1	B5	BD	B9		NV
STA	store A		8D	85	81	91	95	9D	99		
LDX	load X	A2	AE	A6					BE	B6	NV
STX	store X		8E	86						96	
LDY	load Y	A0	AC	A4			B4	BC			NV
STY	store Y		8C	84			94				

<u>Mnemonic</u>	<u>Explanation</u>	<u>Mode implied</u>	<u>Flags</u>
TAX	transfer A to X	AA	NV
TAY	transfer A to Y	A8	NV
TXA	transfer X to A	8A	NV
TYA	transfer Y to A	98	NV

Moving Data between Memory, X, Y and A

Figure 8.1

later.

Now let's review the "moving data" coding decisions in the "good listener" program, Figure 7.3.

In the first instruction, index register X is cleared to zero by loading a zero valued byte from memory.

The opcode identifies the addressing mode as immediate mode (A2 in the LDX row, Figure 8.1). In this mode, the operand is the byte following the opcode. It is known as an immediate operand. Note from Figure 8.1 that registers A and Y can also be loaded with immediate operands. There are no store-into-immediate mode operations, so immediate bytes are considered to be constants. Memory locations in which a program stores data represent variables in the algorithm, quantities that can change during its execution. In the assembler input, an immediate operand is identified by the character '#' preceding the constant. Identify the other immediate operands in the program.

We must defer explaining the second instruction, the JSR, but its effect here is to place the ASCII code for the next key depressed into the accumulator A.

The natural thing to do is to store from A into the display line. The addressing mode of the STA instruction is absolute, indexed by X. Absolute mode means that a full 16-bit address follows the opcode. For reasons we will cover later, the absolute address bytes are reversed in the instruction. The address intended is the start of the display

line, D146₁₆. To change the address of the display line on the screen in a consistent manner, change the absolute addresses starting at hexadecimal addresses 6, 14, 17 and 20. In the assembly language operands, the characters ',X' denote indexing by X, which means that the effective address is the sum of the absolute address and the contents of register X. What happens to the effective address as X is incremented with each repeat of the loop?

There is an instruction in the program using an absolute addressing mode without indexing. It is the JMP instruction at 22₁₆. Guess what JMP stands for? In assembly language, names like REPEAT, FILL and MOVE are created by the programmer to stand for addresses of instructions or data locations. Such names are called labels. Each label appears to the left of the opcode mnemonic and in the symbolic form of instructions in which the address affects the assembled code. Labels may be used whenever they are needed.

Continuing with the coding decisions of "good listener", there is a problem in the second loop with having the key code in register A. We would like to use the sequence

```
MOVE LDY $D147,X
      STY $D146,X
```

to accomplish the left face, forward march of the display. The problem is that STY has no absolute indexed addressing mode. No such opcode in Figure 8.1, right? If we used register Y instead of register X as the index register, would we have the same problem? You bet. To get around the problem, the program saves the key code in the Y register and uses

the accumulator A for the leftward bunny hop. Could this little inelegance be avoided by bunny hopping first, then getting the new key? The answer is no. Figure out why, then test your theory by arranging the program that way and observing what happens.

Do you get the impression that coding can be fun? Let's see if you can reverse the roles of X and Y in "good listener" and get a working program. To compare contents of Y with an immediate line length, the instruction reads

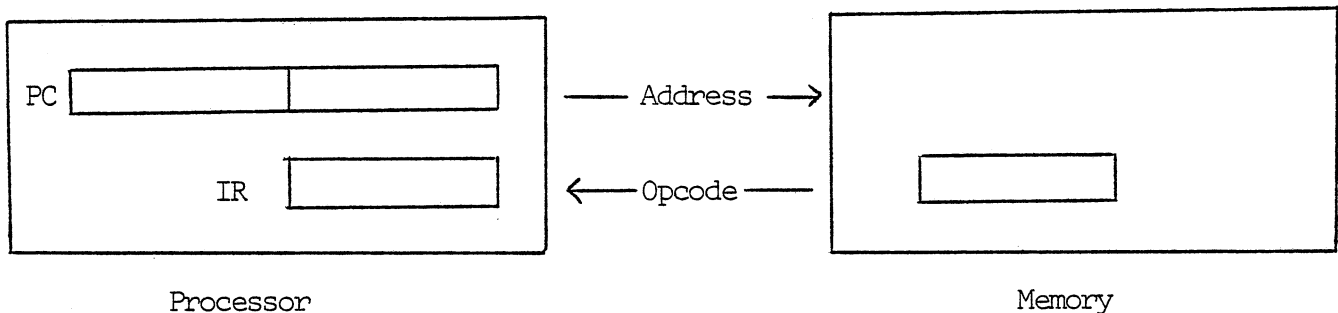
CPY #20;END OF THE LINE?

and the opcode is C0.

9. Fetch and Step It

Many of the 6502 instructions are easier to understand if you know how a computer's processor goes from instruction to instruction during program execution. The processor keeps track of its location in the program by means of a register called the program counter (PC). The program counter is the size of an effective memory address. After the execution of an instruction, it holds the address of the next instruction to be executed.

Each instruction is processed as it is encountered, in a two-part cycle, the fetch-and-execute cycle. In the first part,



the program counter contents are sent to the memory unit which sends the opcode back to the processor register called the instruction register (IR). The program counter is incremented by one. This is the fetch cycle. Now the processor determines the operation and address mode and carries out the execution cycle. In the process the program counter advances to the next opcode. Note - program counter contents can be affected by JSR and branch instructions.

It is the program counter that provides access to the immediate operands described in the last section. It contains the effective address at the beginning of the execution cycle. It is efficient to access constants this way, but not variables. For a variable, some additional mechanism would be necessary to get back to the stored value, once the program counter had gone past it. Besides, programs in ROM would have to avoid immediate stores anyway. For these reasons, the immediate addressing mode is for constants only.

In absolute and absolute indexed addressing modes, the program counter is used to fetch a full two-byte address. Because the address bytes are stored in reverse order, the right byte is fetched first. This is handy because in indexing the processor must add a register byte to the absolute address to form the effective address. It gets the right byte first and adds the index register contents to it. The sum can be too large for a byte but there is no real problem. With your knowledge of binary, you can confirm that at worst, only one extra bit is needed. The extra bit is called, appropriately enough, the carry bit. The processor retains the carry bit, advances the program counter and fetches the left address byte, then adds the carry to it forming the full effective address.

Branch or jump instructions provide an effective address which can replace the contents of the program counter. The replacement causes the processor to start on a sequence of instructions at another location in memory.

With the 65V Machine Monitor, you start the execution of a program by entering its start location into the address display and depressing the 'G' key. This transfers the displayed address into the program counter. From that point the processor is at the mercy of the program you keyed in. Program bugs or loading errors may bring the program counter to the address of an invalid or unintended opcode. When that happens, the program loses control of the processor and, if anything occurs next, it isn't what you had in mind. Fortunately, the reset key forces the program counter to the starting address of a program that will listen to you, so that you can get back to the Monitor and look for the problem.

With the program of Figure 9.1, you can execute a series of instructions one at a time and observe their effects on memory and the processor registers. The program contains a "box" of three "no operation" (mnemonic is NOP, pronounced no op, like co-op) opcodes in its middle. A NOP instruction does nothing but advance the program counter by one. To execute one instruction, you insert it in the box where the program counter will reach it as the program executes. Any instruction will fit since none are longer than three bytes, but be sure that the unused box bytes are filled with the EA_{16} opcode of a NOP instruction.

The instructions preceding the box load the processor registers and flags from memory locations $00F0_{16}$ through $00F3_{16}$. After the execution of the inserted instruction, the contents of all processor registers are stored in the corresponding memory locations and the program returns to the address mode of the Monitor. At this point you can examine memory to verify the effects of the executed instruction.

```

;RESTORE PROCESSOR TO EXECUTE BOX INSTRUCTION

00D0 A5 F0          LDA SAVE    ; LOAD FLAGS AND
00D2 48             PHA          ; PUSH THEM ONTO THE STACK
00D3 A6 F1          LDX SAVE+1  ; LOAD X
00D5 A4 F2          LDY SAVE+2  ; LOAD Y
00D7 A5 F3          LDA SAVE+3  ; LOAD A
00D9 28             PLP          ; PULL FLAGS FROM STACK

;THE BOX
00DA EA            BOX NOP      ; A BOX OF THREE BYTES
00DB EA            NOP          ; TO HOLD LARGEST INSTRUCTION
00DC EA            NOP          ; ALWAYS RESTORE TRAILING NOP'S

;SAVE PROCESSOR STATE AFTER EXECUTION
00DD 08            PHP          ; PUSH FLAGS
00DE 86 F1          STX SAVE+1  ; SAVE X
00E0 84 F2          STY SAVE+2  ; SAVE Y
00E2 85 F3          STA SAVE+3  ; SAVE A
00E4 68            PLA          ; PULL FLAGS
00E5 BA            TSX
00E6 86 FF          STX SAVE+4  ; SAVE STACK POINTER
00E8 85 F0          STA SAVE    ; SAVE FLAGS
00EA 4C 47 FE       JMP $FE47  ; RETURN TO MONITOR ADDRESS MODE

```

Execute-in-a-Box Program

Figure 9.1

Where do you look for the contents of X, Y and A?

When you have looked around enough you are ready to put the next instruction in the box. At the beginning of a sequence of instructions, you may want to set up initial values for the registers by setting the corresponding memory locations.

One restriction you must observe: avoid placing a branch or jump instruction in the box. Such an instruction would allow the processor to escape from the box, with unpredictable results. No problem though. Branches and jumps effect only the sequencing of instructions, which you are handling anyway, so you would leave them out of any sequence of instruction you were executing step by step.

We are not ready to fully explain the "execute-in-a-box" program now, but one important feature we can consider now is the zero page addressing mode used to access the data beginning at $00F0_{16}$ (SAVE).

Think of memory (RAM) as divided into blocks of 256 bytes each. These blocks are called pages. The left byte of an effective address can be considered as identifying a page, and the right byte as identifying a location within the page. The first page in memory has the identifying number 00 (zero) and is therefore known as the zero page. By having an opcode for a zero page addressing mode, the processor can be told to supply a zero for the left byte of the effective address and the zero byte can be left out of the program. This saves memory space and the execution cycle time it would take to fetch the zero byte from memory. This feature makes zero page memory locations very valuable. The zero page indexed addressing modes work

just as you would suspect. The carry from the indexing sum is added to the zero byte supplied by the processor to form the left byte of the effective address.

Load the program and try out a sequence or two. It may be your best friend when you are trying to find out what is going on in some other program.

10. Start Waving Your Flag

Decisions are implemented in programs by branch instructions. In these instructions, the test of a condition determines whether or not the program counter is loaded with a new value. In the 6502, branching is controlled by a set of one-bit registers called flags. The important flags for branching are given the symbolic names N, V, Z and C by 6502 programmers. They are known as the negative (N), overflow (V), zero (Z), and carry (C) flags.

Flags are set (to 1) and cleared (to 0) by the action of instructions. In fact, programmers need to know how each instruction affects the flags. Look at the Figure 8.1 summary of the 6502 move instructions. The letters at the right indicate when the corresponding flag is affected by the instruction. If you know that an operation affects a certain flag, you'll know whether it sets or clears the flag, because flags have consistent, easy-to-learn meanings. Interpret the N, V and Z flags as follows:

N = 1 means the result is negative (0 means positive or zero)

V = 1 means the result is invalid (more about this later)

Z = 1 means the result is zero.

The carry flag represents the carry bit or some other one-bit extension of the result. As we go through more of the instruction set you will see how useful the carry bit extension is.

Figure 10.1 summarizes the 6502 branch instructions. The conditional branch instructions check for a particular flag condition

<u>Mnemonic</u>	<u>Explanation</u>	<u>Opcode</u>
BCC	branch if carry is clear (C = 0)	90
BCS	branch if carry is set	B0
BEQ	branch if equal (Z = 1)	F0
BNE	branch if not equal (Z = 0)	D0
BMI	branch if minus (N = 1)	30
BPL	branch if plus	10
JMP	unconditional jump	4C absolute mode 6C indirect mode

Address Modes

		immediate	absolute	zero page	(ind,X)	(ind),Y	zero page,X	abs,X	abs,Y	N
CMP	set flag by A - Memory	C9	CD	C5	C1	D1	D5	DD	D9	
CPX	set flag by X - Memory	E0	EC	E4						
CPY	set flag by Y - Memory	C0	CC	C4						

6502 Branch and Compare Instructions

Figure 10.1

and loads the program counter if it is met. If the condition is not met, no branch occurs. The program counter simply advances to the next opcode. The conditional branches use the relative address mode. In this mode, a full effective address is loaded into the program counter with a new value formed from a single byte following the opcode. This is done by adding the single byte to the program counter. A negative instruction byte allows the processor to branch backwards in the program, as you can see happening in "good listener". More details on relative addressing will come up in the next section.

The mnemonic JMP is used for an unconditional branch or jump. No flag testing is involved and the branch is taken every time. Absolute, rather than relative, addressing mode is used. For an example, see "good listener". The Monitor's machine code provides a good illustration of the JMP indirect mode shown in Figure 10.1. In indirect addressing, the address provided by the instruction is used to fetch the address of the operand. In the case of the JMP, the operand is the jump destination, the address to replace the program counter. When you depress the 'G' key, the Monitor branches to the machine language instruction 6C FE 00. Can you figure out what is in locations 00FE₁₆ and 00FF₁₆ when the Monitor is running? Elementary, my dear Watson.

Some instructions are devoted to setting flags and do nothing else. They are called "compare" instructions. In the 6502 instruction set, compare instructions subtract a memory byte from the contents of a processor register and set the flags according to the result leaving the operands unchanged. The 6502 compares are included in Figure 10.1. Many decisions represented by diamond boxes in flowcharts are

implemented by a compare instruction, followed by a conditional branch.

When the distance between a branch instruction and the intended branch destination is too great for one byte, the JMP instruction with its absolute address mode can come to the rescue. For example, in

```
CMP  #_____  
BEQ  AWAY,
```

if the location represented by AWAY is too far away, then use

```
CMP  #_____  
BNE  SKIP  
JMP  AWAY  
SKIP
```

There are all sorts of interesting variations to be made in "good listener", based on recognizing a particular character as a command. For example, one could have "good listener" return to the address mode of the Monitor on command. In the Monitor you could change the location or length of the display line, then restart "good listener". This version could be used to compose character pictures on the screen. For your JMP into the Monitor, $FE43_{16}$ is a good address, but remember to reverse the bytes. For the compare instruction to recognize the new command you'll need the ASCII code for the command's key. If necessary, you can call upon the binary display program of Section 4, or consult the appendix.

Another idea is to have "good listener" advance the display area to a new line, all by itself, on command. After all, the line location in the program is in RAM when the program is loaded and can be treated as variable data.

When making a change in a program, the insertion of instructions changes the location of instructions following the insertion point. Most instructions are not affected by such relocations, but some are. Absolute address mode instructions may have to be adjusted. In relative addressing, any insertion or deletion between a branch instruction and its branch destination effects the displacement byte which follows the opcode. Watch your step when making changes. Bugs are easier to prevent than to find.

When executing in a box with the program of Figure 9.1, you can follow the changes in the flags. The program restores and saves all flags in location $00F0_{16}$, in the binary format

For example, if the Monitor command `.00F0` displays A6 in the data display, then the binary code

1010 0110

reveals that $N = 1$, $V = 0$, $Z = 1$, $C = 0$ at that point.

11. A Total Mystery

How about a little challenge? Load the following program, double check the loading and start the execution. You should

<u>Location</u>	<u>Program</u>
.0000/	D8*A9*00*85*FF*85*FE*85*
0008	FC*20*ED*FE*C9*2B*D0*08*
0010	A5*FE*65*FC*50*34*70*0B*
0018	C9*2D*D0*24*38*A5*FE*E5*
0020	FC*50*27*A2*03*B5*51*9D*
0028	D0*D0*CA*10*F8*20*ED*FE*
0030	C9*0D*D0*F9*A2*03*A9*20*
0038	9D*D0*D0*CA*10*FA*30*C1*
0040	20*93*FE*30*C4*A2*00*20*
0048	DA*FE*85*FE*20*AC*FE*D0*
0050	B8*54*49*4C*54
.0000 G	

get a display of zeros. Now key-in data and try to discover what the program does. Here are vital clues: the program reacts only to the hexadecimal digits 0-F, the signs '+' and '-', and the 'RETURN' key. The 'RETURN' key is ignored at one point but is vital at another.

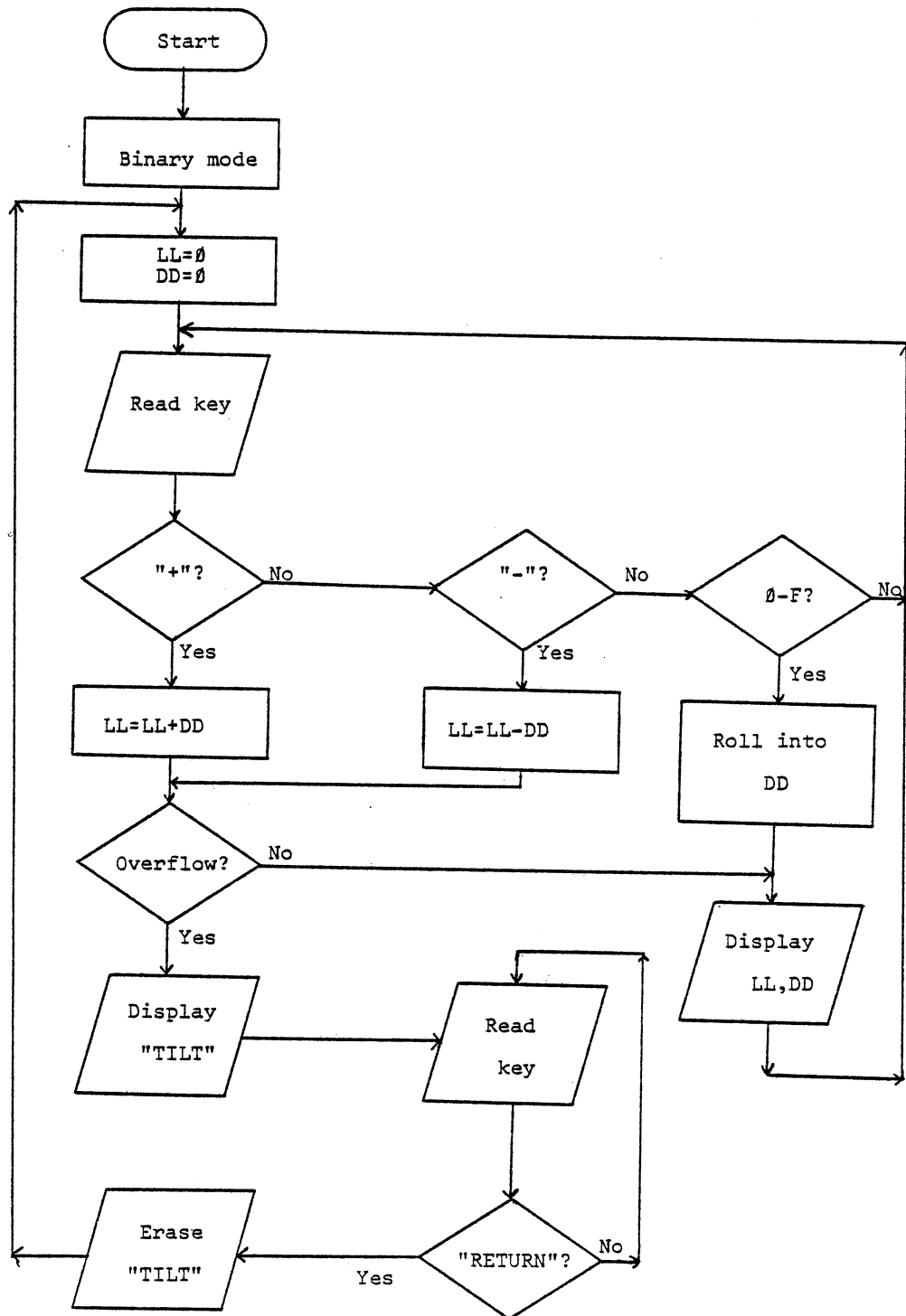
You will find an explanation of the program, starting on the next page, but don't spoil the fun by peeking. Try all sorts of input, watch and record what happens, make guesses and test them. There is a special message which can appear but it goes away. The program loops forever.

Bumfuzzled? If so, here is some information that may clear things up. Do the results you cannot explain involve data having a left hexadecimal digit of eight or greater? To the 6502 processor, such binary codes represent negative numbers. Try starting with a zero and subtracting a positive value, one that reads \$7F (a "\$" preceding a number indicates hexadecimal) or less. The result represents the negative number of the same size. Perhaps now you can explain everything the program is doing, before looking at the flowchart of Figure 11.1, which reveals all. Try to determine the largest positive sum and the smallest negative sum that can be obtained.

There are many ways to represent negative numbers in binary codes. For binary integer arithmetic, most computers use the system that the 6502 does, the one illustrated by the mystery program. It is called two's complement representation. To change the binary representation of a number to the two's complement negative, you first change every bit to its opposite value. This is referred to as complementing the bits. Then add 1 (one) to the result. Since the process amounts to changing the sign of a number, repeating it should produce the original binary code. Does it?

Two's complementing can be done directly in hexadecimal. Replace each hex digit by a complement digit that you obtain by subtracting the original digit from 15_{10} . This does the complementing. Then add 1 (one) to the result.

Trace through the flowchart of Figure 11.1 and try out any parts you did not get into with your blind exploration of the program. The



Flowchart for "Total Mystery"

Figure 11.1

flowchart shows something important about two's complement representation. There are no tests in the algorithm to determine the sign of the data values. With two's complement representation, positive and negative values are processed in exactly the same way.

Did you get the 'TILT' message? It appears when the overflow flag is set by the add or subtract operations. In either case, the message means that the correct result cannot be represented in one byte.

Now that we have reached two's complement, we can understand the 6502 backward branches in relative addressing. In the Figure 7.3, coding of "good listener", the test for the end of the loop is

```
BNE  FILL
```

The reference to the label FILL is assembled as \$F5, the two's complement representation of a negative displacement. In the execution of the instruction, the program counter is first advanced to \$000D, then the negative byte \$F5 is expanded to a 16-bit representation of the same number and is added to the updated value of the program counter to produce

$$\begin{array}{rcl}
 000D_{16} & & \$000D \\
 + FFF5_{16} & \text{or} & + \$FFF5 \\
 \hline
 0002_{16} & & \$0002
 \end{array}$$

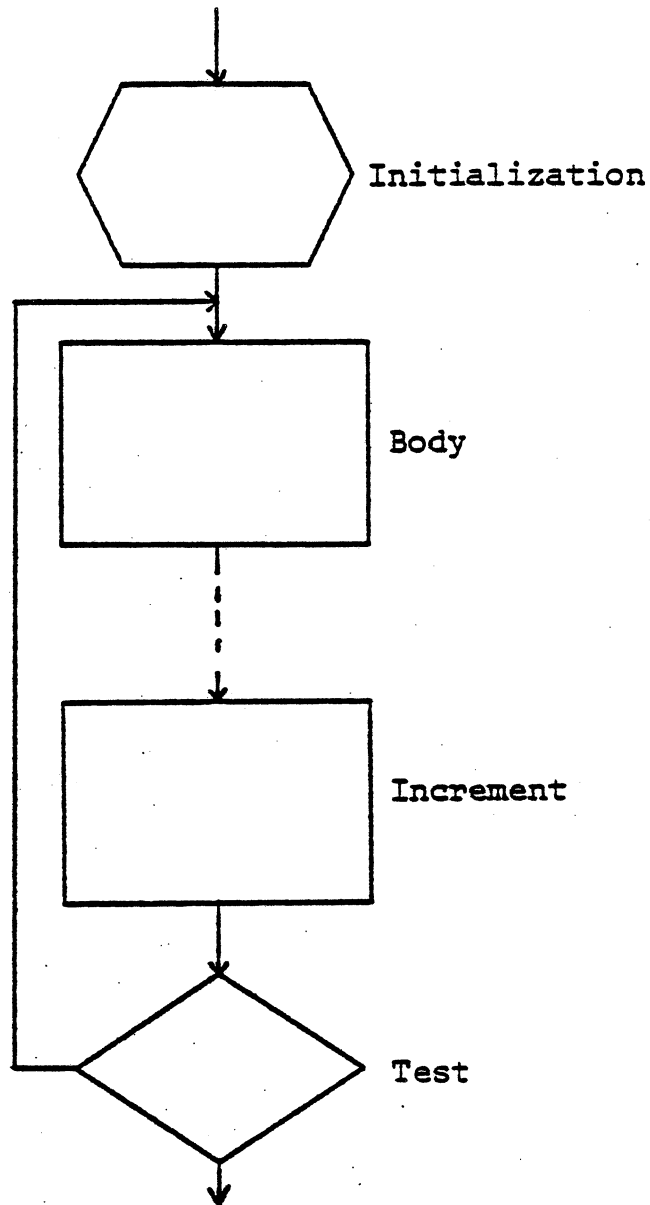
as the branch destination. The addition is ordinary garden-variety binary addition whether the displacement is negative or positive. Does the backward branch to MOVE work the same way?

12. A Loop You Can Count On

In the "good listener" code of Figure 7.3, there are three loops. One pair of loops is nested, one within the other. Two of good listener's loops are controlled by counters to execute a prescribed number of times. The index register X is used as the loop counter within each of these counting loops.

Figure 12.1 shows the usual arrangement of the parts of a counting loop. The body of the loop is the part that is repeated. It is a set of instructions that may contain other loops. The loop counter is given an initial value outside of the loop. Within the loop it is incremented or decremented until it reaches some test value. The test value may be a constant or a variable, depending on when the number of times through the loop is determined.

Many a program bug has hatched when the programmer forgot to initialize the loop counter or chose the wrong test value or branch instruction, causing the loop body to be executed the wrong number of times. A wise precaution is to double check immediately after coding a loop to see what values the loop counter will have on the first and last executions of the body. If undetected, this kind of bug crawls off into some other part of the program and causes something apparently unrelated to go awry. A good way to save yourself a lot of effort looking for bugs of this type is to make it a practice to play processor with a new program, going through it instruction by instruction, calculating the changes that take place and looking for surprises. Programmers call this desk checking because it is done at the desk and not on the computer. You have "execute-in-a-box" to play



Parts of a counting loop

Figure 12.1

processor with.

The 6502 instructions for incrementing and decrementing are shown in Figure 12.2. The index registers X and Y make the best loop counters, but a way is provided to use a memory cell as a counter as well. The reason is that with loops nested within one another, so that several are repeating at one time, an index register would not be available to control each loop. X and Y are preferred as loop counters, primarily because of their ability to access data through indexing. As loop bodies are repeated, the algorithm is often moving through blocks of data in a systematic way. Why do you suppose that increment and decrement instructions are not provided for the accumulator? Generally, it would be occupied with something in the body of the loop and not be available as an index register or loop counter.

You probably noticed that the increment and decrement instructions affect the flags N and Z. This makes it possible to detect when a loop counter reaches a zero or negative value without a compare instruction. When you have a choice, it may be better to count backwards to take advantage of this feature. As an example, look at the program of Figure 12.3 which displays all the graphic symbols with a given high (left) hexadecimal digit in their codes. With this program you can rapidly survey the graphic symbols available to your computer.

In the algorithm, it doesn't matter which order the graphic codes are loaded into display locations, so they are loaded in right-to-left, in decreasing order. The BPL instruction allows a

		absolute	zero page	zero page,X	absolute,X	Flags
DEC	Decrement memory by 1	CE	C6	D6	DE	N,Z
INC	Increment memory by 1	EE	E6	F6	FE	N,Z
implied						
DEX	Decrement X by 1	CA				N,Z
DEY	Decrement Y by 1	88				N,Z
INX	Increment X by 1	EB				N,Z
INY	Increment Y by 1	C8				N,Z

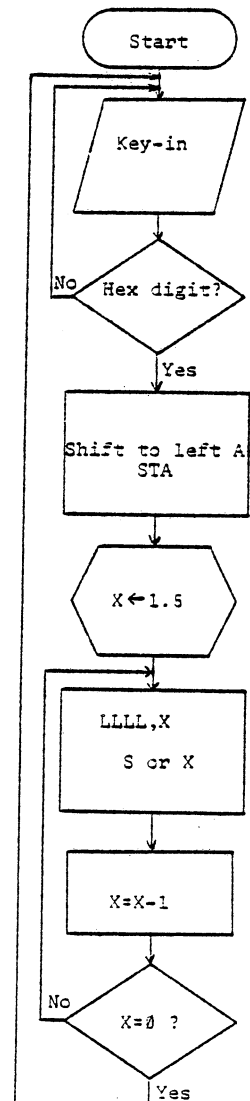
Counter Increment and Decrement Instruction

Figure 12.2

```

;USER KEYS HEX DIGIT D PROGRAM DISPLAYS ALL GRAPHICS DK
;WHERE K IS A HEX DIGIT
0000 20 ED FE BEGIN JSR $FEED ; GET A KEY
0003 20 93 FE FSR $FE93 ; STRIP TO HEX DIGIT
0006 30 F8 BMI BEGIN ; IGNORE NON-DIGITS
0008 0A ASL A ; SHIFT TO LEFT INTO HIGH DIGIT
0009 0A ASL A
000A 0A ASL A
000B 0A ASL A
000C 85 1B STA TEMP ; SAVE
0003 A2 0F LDX #15 ; INITIALIZE COUNTER
0010 8A LOOP TXA ; COUNT TO A FOR LOW DIGIT
0011 05 1B ORA TEMP ; APPEND HIGH DIGIT
0013 9D 46 D1 STA $D0C6,X ; STORE IN DISPLAY LINE
0016 CA DEX ; SEE COMMENTS
0017 10 F7 BPL LOOP ; ON THIS BRANCH
0019 30 E5 BMI BEGIN ; AND THIS DWG
001B TEMP

```



A graphics display program

Figure 12.3

repeat of the body of the loop with $X = 0$. A BNE instruction would make the $X = 1$ iteration the last. The BMI instead of a JMP is a trick to save a byte of program. TEMP is assigned a byte at the end of the program instructions.

If you are looking for a programming challenge involving counting loops, you could attempt a program to produce the same output as this one, but with the displayed symbols arranged in a table of four rows by four columns, with single blank spaces between all of the symbols.

13. A Stack of Letters

Beware. You are in Section 13. To avoid bad luck while in this section, spell 'abracadabra' backwards. Use the program of Figure 13.1. This program has to save a word in memory without letting you see it until you depress the space bar. Then it moves the word into the display area in reverse order. Without modification, the program allows up to 20 characters in an input word.

This little trick can be coded a number of ways on the 6502, but our program uses a feature of this processor that we have not yet considered, the stack pointer. A stack is a type of data structure, an arrangement of data that provides access to an item of data in a particular way. The stack provides last-in first-out (LIFO) access, meaning that it makes available one item at a time, and the available item is the latest one that was placed on the stack. Take that item from the stack and the previously entered item becomes available.

In computer terminology, the available item is called the top of the stack. The operation of placing an item on the stack is a push; removal of an item is a pop or a pull. The backwards speller pushes characters on the stack as you enter them, then pops them all when it sees a blank. They come out in reverse order.

In the 6502, each item on the stack is a byte of information. The data on the stack is not kept in the processor but in memory. The processor has a one-byte register called the stack pointer, S. The stack pointer contains the location of the top of the stack (this location is limited by hardware to page one of memory). A value of 1

;BACKWARDS SPELLER: ENTER A WORD, FOLLOWED BY A BLANK.

;THE WORD THEN APPEARS, BACKWARDS.

;TO ERASE, PRESS ANY KEY, THEN START A NEW WORD.

```
00 A2 40      GO      LDX #64      ; LONGEST WORD
02 A9 20              LDA #C'
04 9D 45 D1 BLA      STA $D145,X ; BLANK OLD WORD
07 CA              DEX
08 D0 FA              BNE BLA      ; TEST LEAVES X=0
0A 20 ED FE BUILD JSR $FEED      ; GET A LETTER
0D C9 20              CMP #C'      ; IF A BLANK
0F F0 04              BEQ DUMP      ; WORD ENDS
11 48              PHA              ; IF NOT, PUSH IT
12 E8              INX              ; COUNT PUSHES
13 D0 F5              BNE BUILD      ; RETURN FOR ANOTHER
15 A0 00      DUMP LDX #0          ; INDEX FRONTWARDS
17 68      POP      PLA              ; POP A LETTER
18 99 46 D1              STA $D146,Y ; SPELL IT OUT
1B C8              INX              ; NEXT LETTER POSITION
1C CA              DEX              ; COUNT POPS
1D D0 F8              BNE POP
1F 20 ED FE              JSR $FEED      ; DELAY BLANKOUT UNTIL ANOTHER KEY
22 4C 00 00              JMP GO          ; IS PRESSED ON RELEASE, REPEAT
```

The Backwards Speller

Figure 13.1

(one) is always used as the page number in stack operations, so the stacked data is always contained in page one of memory or the second block of 256 bytes. A push involves writing the byte into the page one memory byte specified by the stack pointer and decrementing the stack pointer. A pull involves incrementing the stack pointer and reading the byte from the page one memory byte specified by the stack pointer.

Figure 13.2 shows several 6502 stack instructions. The first group is used to access the stack. In the second group, the TXS instruction is the means provided to set the stack pointer to its initial value. In your system, the Monitor has taken care of that, so a TXS need not appear in your programs. Whenever a program uses the stack, it should pull from the stack just what it pushed, no more and no less. This leaves the stack in its beginning state. The backwards speller uses a loop counter to see that this is done.

The "execute-in-a-box" program of Section 9 takes advantage of available stack operations to keep track of changing flags. The set of flags is considered collectively as a register, P, which can be pushed or pulled from the stack. That represents the 6502's only access to the flags as a group. Once on the stack, a pull into the accumulator is a means of transferring the flags to a memory cell for your inspection.

You are allowed to use push and pull operations within the "execute-in-a-box" program because the segments of the program before and after the box leave the stack as they found it. The TSX instruction is used after the contents of X have been saved, to allow you to follow changes in the stack pointer contents.

		<u>Implied</u>	<u>Flags</u>
PHA	Push A onto the stack	48	None
PHP	Push flags onto the stack	08	None
PLA	Pull from the stack into A	68	N,Z
PLP	Pull from the stack into flags	28	All
TSX	Copy stack pointer into X	BA	N,Z
TXS	Copy X into stack pointer	9A	None

Explicit 6502 Stack Operations

Figure 13.2

14. It's The Same Old Routine

You must have noticed the same instruction occupying an important spot in "good listener", the graphics display and backwards speller, namely

```
JSR  $FEED
```

From the remarks in the programs, the effect of the instruction appears to be to put the ASCII code for the next key depressed into the accumulator. Have you tried executing this instruction in a box yet? When you put the code 20 ED FE in the box and execute the program, there is no return to the Monitor, as usual. That is, not until you depress a key and release it. It's an extraordinary instruction that can make the processor wait around all day until you depress that key, isn't it?

To tell the truth, the JSR doesn't do all this. Rather, it branches to a sequence of instructions that does. The set of instructions is called a closed subroutine or simply, a subroutine. The processor begins to execute code starting at \$FEED, continuing until it encounters the opcode \$60. Then, magically enough, it branches right back to the instruction following the JSR instruction. With this information and the help of Figure 14.1, you can do some detective work and figure out what the subroutine at \$FEED really does.

Figure 14.1 is a disassembly table. Take the hexadecimal code in a location and select a row in the table by the left digit and a

OPCODE TABLE

MSD	LSD	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	BRK		ORA-IND, X				ORA-Z, Page	ASL-Z, Page		PHP	ORA-IMM	ASL-A			ORA-ABS	ASL-ABS	
1	BPL		ORA-IND, Y				ORA-Z, Page, X	ASL-Z, Page, X		CLC	ORA-ABS, Y				ORA-ABS, X	ASL-ABS, X	
2	JSR		AND-IND, X			BIT-Z, Page	AND-Z, Page	ROL-Z, Page		PLP	AND-IMM	ROL-A		BIT-ABS	AND-ABS	ROL-ABS	
3	BMI		AND-IND, Y				AND-Z, Page, X	ROL-Z, Page, X		SEC	AND-ABS, Y				AND-ABS, X	ROL-ABS, X	
4	RTI		EOR-IND, X				EOR-Z, Page	LSR-Z, Page		PHA	EOR-IMM	LSR-A			EOR-ABS	LSR-ABS	
5	BVC		EOR-IND, Y				EOR-Z, Page, X	LSR-Z, Page, X		CLI	EOR-ABS, Y				EOR-ABS, X	LSR-ABS, X	
6	RTS		ADC-IND, X				ADC-Z, Page	ROR-Z, Page		PLA	ADC-IMM	ROR-A		JMP-IND	ADC-ABS	ROR-ABS	
7	BVS		ADC-IND, Y				ADC-Z, Page, X			SEI	ADC-ABS, Y				ADC-ABS, X		
8			STA-IND, X			STY-Z, Page	STA-Z, Page	STX-Z, Page		DEY		TXA		STY-ABS	STA-ABS	STX-ABS	
9	BCC		STA-IND, Y			STY-Z, Page, X	STA-Z, Page, X	STX-Z, Page, Y		TYA	STA-ABS, Y	TXS			STA-ABS, X		
A	LDY-IMM		LDA-IND, X	LDX-IMM		LDY-Z, Page	LDA-Z, Page	LDX-Z, Page		TAY	LDA-IMM	TAX		LDY-ABS	LDA-ABS	LDX-ABS	
B	BCS		LDA-IND, Y			LDY-Z, Page, X	LDA-Z, Page, X	LDX-Z, Page, Y		CLV	LDA-ABS, Y	TSX		LDY-ABS, X	LDA-ABS, X	LDX-ABS, Y	
C	CPY-IMM		CMP-IND, X			CPY-Z, Page	CMP-Z, Page	DEC-Z, Page		INY	CMP-IMM	DEX		CPY-ABS	CMP-ABS	DEC-ABS	
D	BNE		CMP-IND, Y				CMP-Z, Page, X	DEC-Z, Page, X		CLD	CMP-ABS, Y				CMP-ABS, X	DEC-ABS, X	
E	CPX-IMM		SBC-IND, X			CPX-Z, Page	SBC-Z, Page	INC-Z, Page		INX	SBC-IMM	NOP		CPY-ABS	SBC-ABS	INC-ABS	
F	BEQ		SBC-IND, Y				SBC-Z, Page, X	INC-Z, Page, X		SED	SBC-ABS, Y				SBC-ABS, X	INC-ABS, X	

LSD - Least Significant Digit

MSD - Most Significant Digit

6502 Disassembly Table

Figure 14.1

column by the right digit. If the code is an opcode, the row and column intersection will show the operation and the addressing mode. What makes it fun is that now you must find which bytes represents the next opcode, and the correct interpretation of the bytes in between. Happy hunting!

Where the subroutine at \$FEED appears to be reading a memory location, it is not a memory location at all. It is what is called a port, a connection to an input or output device. In this case, it is an input device, the computer's keyboard. There is a signal transmitted to the port when a key is depressed. When all keys are released, the data received through the port is loaded into the accumulator.

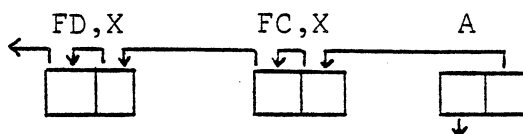
A program containing a JSR is said to call the subroutine starting at the branch address in the instruction. The JSR makes provisions for returning the processor to the calling program. It does so by pushing the value contained in the advanced program counter onto the stack before replacing it to execute the branch. The pushed address is called the return address. A subroutine may contain any number of return instructions, written as a mnemonic RTS in the symbolic form of the subroutine. When an RTS (60_{16}) is encountered, the return address is pulled from the stack into the program counter producing the branch to the point immediately after the JSR instruction.

When are subroutines useful? Primarily, when the same processing is needed in more than one place. "Good listener", for example, reads the next key-in in two places. The single copy of the processing code

is executed wherever it is needed. Subroutine calling is an important feature that multiplies the power of computer systems. Collections of subroutines can be built to carry out frequently used functions in users' programs.

Figure 14.2 describes some of the subroutines in the Monitor. The display subroutine at \$FEAC can be used to make an improvement in "execute-in-a-box" without adding much to its length. When storing register contents in \$00F0 through \$00F4, also store key register contents in \$00FC, \$00FE and \$00FF. Or, replace a key memory location in the simulated program by one of these addresses. When "execute-in-a-box" returns to the Monitor, a call to \$FEAC is the first instruction executed, so the display of desired trace data is automatic.

<u>Entry</u>	<u>Effect Registers</u>	<u>Effect</u>
\$FE93	A	Returns binary value of hexadecimal digit, N=0 If not a digit, returns \$80 and N=1
\$FEAC	A,X,Y	Displays as hexadecimal digits LLLL DD the contents of 00FF and 00FE (LLLL) and 00FC (DD)
\$FECA	A,Y	Stores the ASCII code for the right hexadecimal digit A in LLLL,Y and increments Y by 1
\$FEDA	A,Y	Shifts right digit of A into 00FD,X and 00FC,X as shown. Clears A,Y



\$FEE9	A	Get next ASCII character from keyboard or UART, depending on location 00FB
--------	---	---

Some Useful Monitor Subroutines

Figure 14.2

15. Extending The Mystery

It's time to come forward with the whole story on the "total mystery" program of Section 11. Since "total mystery" calls many monitor subroutines, it seemed only right to keep its workings under wraps until the subroutines were explained. Now Figure 15.1 exposes all of total mystery's secrets.

The heart of "total mystery" is the add and subtract instructions ADC and SBC. Since they are the basis of all arithmetic processing on the 6502, an ample set of addressing modes is provided for these instructions, as seen in Figure 15.2. Addition and subtraction are carried out in the 6502 by loading the accumulator, then using ADC or SBC to form the sum or difference in the accumulator. The carry flag gets in on the act: ADC sums accumulator, operand byte and carry flag. SBC subtracts the operand from the accumulator and adds in the complement of the carry as a "borrow". To get a correct one-byte addition, you must make sure that the carry flag is cleared beforehand. An instruction CLC is available to do this. It was not necessary in "total mystery" because the compare instruction at \$001C clears the carry flag. A correct, one-byte subtraction requires setting the carry flag so that no borrow is assumed. The SEI instruction of Figure 15.2 serves that purpose.

The reason for involving the carry flag in ADC and SBC is to enable addition and subtraction to be extended to numbers larger than one byte. Starting at the rightmost byte of the numbers, the value resulting from each addition or subtraction represents the carry or borrow required into the next byte to the left.

```

;TOTAL MYSTERY
00 D8      BEGIN    CLD      ; CLEAR DECIMAL MODE FOR BINARY
                                OPERATION
01 A9 00      REPEAT  LDA     #0      ; CLEAR LLLL, DD DISPLAY
03 85 FF      STA     $FF
05 85 FE      STA     $FE
07 85 FC      STA     $FC
09 20 ED FE GET JSR     $FEED      ; GET KEY
0C C9 2B      CMP     #$2B      ; A '+'? CLEARS CARRY
0E D0 08      BNE     NEG      ; NO, TEST FOR MINUS
10 A5 FE      LDA     $FE      ; YES
12 65 FC      ADC     $FC      ; ONE-BYTE ADD
14 50      34      BVC     STORE      ; NO OVERFLOW, STORE AND DISPLAY
16 70 0B      BVS     TILT
18 C9 2D      NEG      CMP     #$2D      ; A '-'?
1A D0      24      BNE     DIG      ; NO, TEST FOR DIGIT
1C 38      SEC      ; SET CARRY
1D A5 FE      LDA     $FE      ;
1F E5 FC      SBC     $FC      ; ONE-BYTE SUBTRACT
21 50      27      BVC     STORE      ; NO OVERFLOW, STORE AND DISPLAY
23 A2 03      TILT   LDX     #3
25 B5 51      MLP      LDA     MESS,Y      ; WRITE MESSAGE TO DISPLAY
27 9D D0 D0    STA     4D0D0,X
2A CA      DEX
2B 10 F8      BPL     MLP
2D 20 ED FE WAIT JSR     $FEED      ; WAIT FOR RETURN KEY
30 C9 0D      CMP     #$0D
32 D0 F9      BNE     WAIT
34 A2 03      LDX     #3
36 A9 20      LDA     #$20      ; CLEAR MESSAGE
38 9D D0 D0 D0 ERASE STA     $D0D0,X
3B CA      DEX
3C 10 FA      BPL     ERASE
3E 30 C1      BMI     REPEAT
40 20 93 FE DIG JSR     $FE93      ; STRIP TO DIGIT
43 30 C4      BMI     GET      ; IF NOT A DIGIT, IGNORE
45 A2 00      LDX     #0
47 20 DA FE      JSR     $FEDA      ; ROLL DIGIT INTO DD
4A 85 FE      STORE  STA     $FE      ; STORE RESULT IN LLLL+1
4C 20 AC FE      JSR     $FEAC      ; DISPLAY LLLL DD, CLEAR Z
4F D0      B8      BNE     GET
51 54 49      MESS   .BYTE  'TILT'
53 4C 54

```

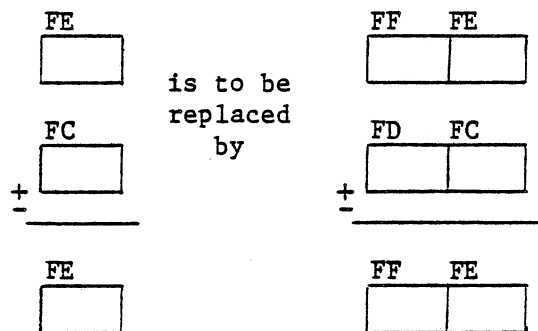
The Total Mystery Program

Figure 15.1

		immediate	absolute	zero page	(ind,X)	(ind),Y	zero page,X	absolute,X	absolute Y	<u>Flags affected</u>
ADC	Add with carry to accumulator	69	6D	65	61	71	75	7D	79	NZCV
SBC	Subtract with carry from accumulator	E9	ED	E5	E1	F1	F5	FD	F9	NZCV
CLC	Clear carry flag	18								C=0
SEI	Set carry flag	78								C=1

Add and Subtract and Carry Flag Set Up

Figure 15.2



After ADC \$FC at 0012, insert:

STA \$FE

LDA \$FF

ADC \$FD

After SBC \$FC at 001F, insert:

STA \$FE

LDA \$FF

SBC \$FD

and change STA \$FE at 004A to

STORE STA \$FF

Extending Total Mystery to Two-Byte Arithmetic

Figure 15.3

Figure 15.3 shows how to extend the "total mystery" program to two-byte arithmetic. The carry produced by adding \$00FE and \$00FC contributes to the sum of \$00FF and \$00FD. The overflow flag has meaning only when the most significant, or leftmost, bytes of the numbers have been processed. The arrangement of the numbers in memory may not seem very natural to you. It is done this way in order to make use of the Monitor subroutines at \$FEAC and \$FEDA. A more natural arrangement is illustrated in the subroutines of Figure 15.4.

;ADD INTEGERS N BYTES LONG, STARTING AT \$E0 AND \$E8
;WORKS FOR N UP TO 8.
;CALL WITH Y=N-1. AFFECTS A.

ADDN CLC

ANLP LDA \$E0,Y

ADC \$E8,Y

STA \$E0,Y

DEY

BPL ANLP

RTS

;UNSIGNED ADD, OF A TO N-BYTE ACCUMULATOR AT \$E0
;CALL WITH X=N-1.

ADD1 CLC

A1LP ADC \$E0,X

STA \$E0,X

DEX

LDA #0

BCS A1LP

RTS

Two Subroutines for N-Byte Addition

Figure 15.4

16. Bit By Bit

The 6502 processor can address bytes, but not bits within a byte. Two groups of instructions handle information at the bit level: shift instructions move bits to where they are needed, and bit logical instructions operate on bits individually.

6502 shift instructions are illustrated in Figure 16.1. The byte contents of the accumulator, or a memory cell, can be shifted one bit position, left or right. The carry flag acts as an extension of the shifted byte, receiving the bit value shifted out of the byte. The ASL and LSR instructions bring a cleared bit into the opposite end of the byte. The rotate instructions bring in the previous value of the carry flag. Generally, "arithmetic" shifts are those that produce multiplication or division by two, for both positive and negative values. Since shifting right to divide by two would not work for two's complement negative values, the right shift is called a logical rather than an arithmetic shift.

The binary display program of Section 4 is based on shift instructions, using them to extract and use one bit of the displayed value at a time. The symbolic form is shown in Figure 16.2.

A logical shift right in the accumulator is central to the subroutine of Figure 16.3. This subroutine could prove useful in many places. For one, use it to improve "execute-in-a-box" so that it displays the flags, X, Y, A and S automatically. You can put the subroutine anywhere without changing any code. Subroutines with this property are called relocatable. The key to writing relocatable

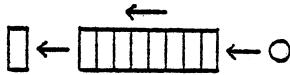
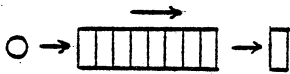
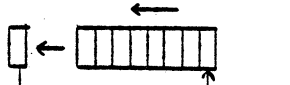
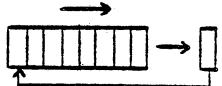
Diagram	Mnemonic	Address Modes					Flags
		Absolute	Zero Page	Accumulator	Z Page, X	Abs, X	
	ASL	0E	06	0A	16	1E	N,Z,C
	LSR	4E	46	4A	56	5E	N=0,Z,C
	ROL	2E	26	2A	36	3E	N,Z,C
	ROR	6E	66	6A	76	7E	N,Z,C

Figure 16.1 Shift instructions

```

;BINARY DISPLAY PROGRAM OF SECTION 4
20 ED FE  LOOP  JSR $FEED  ; GET A KEY
85 F0      STA $F0      ; SAVE IN $F0
A9 18      LDA #$18     ; HEX 30,SHIFTED RIGHT
A2 07      LDX #7
9D D2 D0   OUT  STA $D0D2,X ; SET UP DISPLAY CELL
66 F0      R0R $F0      ; GET NEXT BIT
1E D2 D0   ASL $D0D2,X ; SHIFT BIT IN,PRESERVE CARRY
CA         DEX
10 F5      BPL OUT
30 EA      BMI LOOP

```

Binary Display Program

Figure 16.2

```

;HEX DISPLAY OF ZERO PAGE BINARY CODE STARTING AT X
;DISPLAY STARTS AT $D0C6,Y
;NUMBER OF BINARY BYTES IN A
;USES $FF FOR COUNTER
;
85 FF      HEXTV  STA  $FF      ; BINARY BYTE COUNTER
B5 00      LOOPH  LDA  0,X      ; GET BINARY BYTE
4A         LSR          ; SHIFT LEFT HEX DIGIT
4A         LSR          ; TO RIGHT DIGIT
4A         LSR
4A         LSR
20 CA FE    JSR $FECA ; ASCII TO $D0C6,Y;INCREMENT Y
B5 00      LDA  0,X      ; GET A FRESH COPY
20 CA FE    JSR $FECA ; RIGHT DIGIT ASCII TO $D066,Y AND INC Y
EA         NOP          ; FOR GAP BETWEEN BYTES,INSERT INY HERE
E8         INX          ; ADVANCE IN BINARY INPUT
C6 FF      DEC  $FF      ; COUNTING INPUT BYTES
D0 EC      BNE  LOOPH
           RTS

```

Binary to Hexadecimal Display Subroutine

Figure 16.3

subroutines is to avoid absolute or zero page addressing modes referencing bytes within the subroutine.

Normally you would put a subroutine somewhere beyond the first two pages of memory. Zero page locations are too valuable and the stack occupies page one of memory.

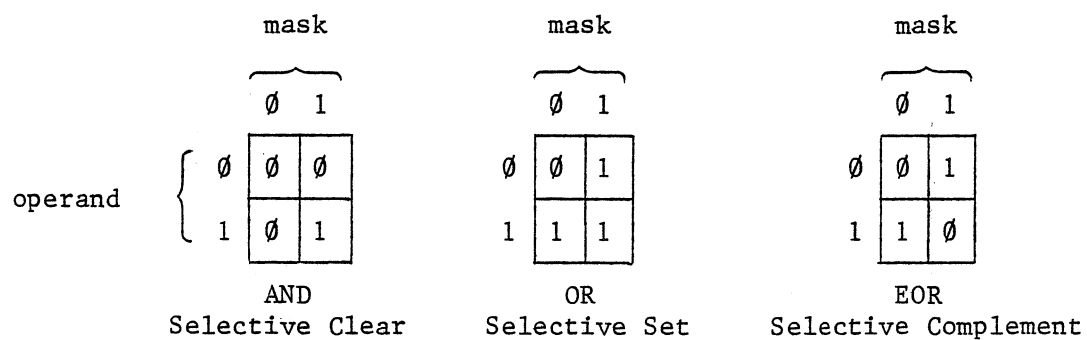
What happens when register Y is incremented beyond 255_{10} or FF_{16} ? It just starts over, right? You can take advantage of this to display several lines in "execute-in-a-box", letting the active line do a 'round robin' progression over the display.

17. It's Very Logical

In most computers the individual bits within addressable units are operated on by a technique known as masking. A binary code called a mask selects the bits to be operated upon. The operations are called bit logical or simply, logical operations. Like most processors, the 6502 provides bit logical operations for selectively clearing, setting and complementing bits.

In the logical operations, each bit of the mask interacts with the corresponding bit of the operand in the same position. The effects on operand bits are shown in Figure 17.1. The names AND and OR are rather universal names for these operations. EOR is short for exclusive or. The AND operation clears all bits in the operand selected by 0's of the mask. The OR sets all bits selected by mask 1's, while the EOR complements them. Figure 17.2 covers the 6502 logical instructions and a special compare instruction (BIT) which uses the accumulator as a mask to select individual bits for setting the zero flag Z.

Logical operations are important in many algorithms because they represent, very neatly, the combining of sets of objects to form new sets. Imagine that we have devoted one bit to each object. Then a set of objects is represented by a binary code in which '1' means 'belongs to the set'. Now the AND of two codes is the intersection of the sets, the objects belonging to both. OR the codes to get the union of the sets, the set of objects belonging to one or the other. EOR produces the union but eliminates objects in both sets.



The Effect of Logical Operations

Figure 17.1

Addressing Modes

		immediate	absolute	zero page	(ind,X)	(ind),Y	zero page,X	abs,X	abs,Y	Flags
AND	AND accumulator and memory mask	29	2D	25	21	31	35	3D	39	N,Z
ORA	OR accumulator and memory mask	09	0D	05	01	11	15	1D	19	N,Z
EOR	EXCLUSIVE-OR accumulator and memory mask	49	4D	45	41	51	55	5D	59	N,Z

BIT	Test memory by accumulator	2C - Absolute	Z = 0 if A AND memory = 0
	mask	24 - Zero page	= 1 otherwise
			N = memory sign bit
			V = next bit of memory

6502 Logical Instructions

Figure 17.2

The Monitor subroutine starting at \$FECA illustrates masking with logical instructions. It is called by the hexadecimal display subroutine of the last section. As Figure 17.3 shows, the subroutine constructs and stores the ASCII code for a hexadecimal digit contained in the right four bits of the accumulator. With your improved "execute-in-a-box", trace through the routine with some typical input data and confirm its behavior.

```

;DISPLAY AT $D0C6,Y THE ASCII CODE
;FOR THE RIGHT NIBBLE OF A
;INCREMENTS Y
;
FECA 29 0F      ASCII AND #$0F      ; REPLACE LEFT NIBBLE
09 30          ORA #$30            ; BY '3'
C9 3A          CMP #$3A            ; WAS IT 0-9?
30 03          BMI STO             ; YES,NO ADJUST NECESSARY
18            CLC                  ;
69 07          ADC #7              ; NO,ADD 7 FOR A-F ASCII
99 C6 D0 STO   STA $D0C6,Y        ; STORE IN DISPLAY LOCATION
C8            INY                  ; ADVANCE TO NEXT DISPLAY LOCATION
60            RTS

```

Monitor Subroutine Featuring Logical Operations

Figure 17.3